Representation and Tractability in Set Theoretic Estimation: Sheaves, Rectangles, and the Coupling Obstruction

Project `thejeb`

July 23, 2026

Abstract

The feasible set of a set theoretic estimation (STE) problem [4] lives in $\mathcal{P}(\Xi)$, and when the hypothesis space is a product over N variables, $|\Xi|$ is exponential in N: representing the feasible set extensionally hits a 2^N wall that is machine-checked in this project (`Ste.DynamicFrame.Model.card_allExactState`). This note asks when the feasible set can be represented *locally* and glued, and answers three pieces of the question with Lean-verified results in the module `Ste.Sheaf`. (i) *Gluing on the constraint side*: the assignment $J \mapsto \mathrm{feas}(J)$ from constraint subsets to partial feasible sets sends unions to intersections (`partialFeasibilitySet_iUnion`), so any cover of the constraint index recovers the global feasible set as the limit of local ones (`feasibilitySet_eq_iInter_cover`); on this side there is no obstruction. (ii) *A tractable upper bound*: if every constraint is variable-separable (a rectangle), the feasible set is itself a rectangle (`rectangular_feasibilitySet`) representable by one subset per variable—$\sum$-space, not $\prod$-space—with product cardinality (`rectangular_encard`). (iii) *The obstruction*: a single coupling constraint, the diagonal on two bits, is provably not a rectangle (`diagonal_not_rectangular`). This is the elementary form of the Abramsky–Brandenburger contextuality obstruction [1], and the same shape as this project's non-transitivity countermodel (`Ste.DynamicFrame.Counterexample`). Four further Lean modules push toward decomposition: `Ste.Decomposition` proves that disconnected constraint blocks give a product feasible set with multiplicative cost (`feasibilitySet_blockFamily`, `encard_feasibilitySet_blockFamily`); `Ste.Support` gives constraints a scope and shows the diagonal's scope is exactly $\{0,1\}$ (`diagonal_support_full`); `Ste.Conditioning` mechanizes variable elimination—conditioning commutes with feasibility, shrinks scopes, preserves rectangles, and, applied to a *cut* variable, disconnects the problem into independent blocks (`piecewise_mem_condition_feasib`) and `Ste.Treewidth` proves the general cut-elimination theorem (`cut_elimination`), the join-scope accounting behind bucket elimination, the quantitative *bag bound*: the table of a constraint on a scope σ has at most $\prod_{v\in\sigma}|A_v| \le k^{w+1}$ rows (`table_encard_le`, `elimination_step`), and the *elimination-order total-space bound*: n bags of width w cost at most $n\cdot k^{w+1}$ total table rows (`elimination_order_total_bound`). `Ste.Elimination` then mechanizes *running* an order: the bucket-elimination fold on lists of scoped constraints is correct (`joinConstraint_bucketEliminate`), a complete order decides the substituted problem (`bucketEliminate_decides`), and a width-w run of n steps materializes at most $n \cdot k^{w+1}$ rows in total (`bucketEliminate_total_space`). `Ste.Treedecomp` discharges the width hypothesis: the *induced treewidth* of an instance is defined as the least achievable elimination width, proven well-defined and *attained* by a concrete optimal order (`achievesWidth_inducedTreewidth`), so some complete order decides the substituted problem in $n\cdot k^{\mathrm{tw}(B)+1}$ total table space *unconditionally* (`bucketEliminate_treewidth_bound`). A breadth-first sweep then closes six further leads, each machine-checked: projective ($\exists$-)elimination decides feasibility *unconditionally* (`projectEliminate_decides_feasibility`); elimination width equals tree-decomposition treewidth in *both* directions (`treewidth_primalGraph_eq`, over a from-scratch Robertson–Seymour tree-decomposition datatype); the block law becomes an n-ary product (`encard_feasibilitySet_blocks`); the variable-side presheaf glues for rectangles and provably fails for the coupling (`rectangular_glue`, `diagonal_gluing_fails`); the Čech obstruction is computed (0 for rectangles, 2 for the diagonal, `diagonal_cechObstruction`); and the

junction-tree representation is polynomial-size and faithful (`junctionTree_size_le`). A second sweep then closes the dichotomy's hard side — singleton-cover gluing succeeds *iff* the constraint is a rectangle (`cechVanishes_iff_rectangular`) and the n-fold coupling has Čech obstruction $\geq 2^n - 2$ against a constant-size feasible set (`cechObstruction_allEqual_ge`), so unbounded coupling is exponential where bounded width is linear — and mechanizes Freuder's width-1 tractability (a nonempty arc-consistent chain is solvable, `arcConsistent_chain_solvable`). The genuine remaining outlook is the general cohomological programme — the full $\check{H}^1$ functor over arbitrary covers and the quantitative obstruction-size claim — together with higher-width k-consistency and efficient computation of treewidth, which is NP-hard [2, 5, 6, 8].

1 The problem: the 2^N wall

Combettes' set theoretic estimation [4] specifies a solution space Ξ, an index set I of pieces of information, and a property set $S_i \subseteq \Xi$ for each $i \in I$; the feasible (solution) set is $S = \bigcap_i S_i$. The framework is silent about *representation*: it treats S as a set, not as a data structure.

The silence becomes a cost the moment Ξ is a product. In the estimation problems this project cares about—dynamic frames over documents, constraint grammars, hyperframe lattices—the hypothesis space factors as $\Xi = \prod_{v \in V} A_v$ over a family of variables (claims, mentions, cells), and $|\Xi| = \prod_v |A_v|$ is exponential in $|V|$. The finite solver in `Ste.DynamicFrame.FiniteSolver` makes the wall precise and machine-checked: over N ambient worlds there are exactly 2^N extensional states (`Model.card_allExactStates`), and crisp STE has full subset expressivity—a single property set can carve out *any* subset of the universe (`arbitrary_subset_as_single_property`)—so without structural assumptions on the constraints, no representation smaller than 2^N can be complete.

The question of this note: *which structural assumptions buy which representations?* The natural language for the question is sheaf-theoretic, because "represent locally and glue" is exactly what a sheaf condition asserts.

2 Gluing over the constraint poset

Fix $S : I \to \mathcal{P}(\Xi)$ and write, for a subset $J \subseteq I$ of the constraint index,

$$\mathrm{feas}(J) \;=\; \bigcap_{i \in J} S_i \qquad (\texttt{partialFeasibilitySet}\ S\ J),$$

the set of hypotheses consistent with the information in J. The assignment $J \mapsto \mathrm{feas}(J)$ is a presheaf on the poset $\mathcal{P}(I)$ (ordered by inclusion, mapping contravariantly): if $J \subseteq K$ then $\mathrm{feas}(K) \subseteq \mathrm{feas}(J)$, which is the information-monotonicity lemma `partialFeasibilitySet_antitone` already verified in `Ste.Basic`.

The new, machine-checked content of `Ste.Sheaf` is that this presheaf satisfies the gluing condition exactly, with no sheafification needed. The engine is the Formal Concept Analysis bridge of `Ste.HyperFrame` [7]: partial feasibility is definitionally the lower polar of the satisfaction context (`partialFeasibilitySet_eq_lowerPolar`), and polars turn unions into intersections.

Theorem 1 (`partialFeasibilitySet_iUnion`)**.** *For any family $\mathcal{J} : \kappa \to \mathcal{P}(I)$ of constraint subsets,*

$$\mathrm{feas}\Big(\bigcup_k \mathcal{J}_k\Big) \;=\; \bigcap_k \mathrm{feas}(\mathcal{J}_k).$$

Theorem 2 (`feasibilitySet_eq_iInter_cover`)**.** *If $\bigcup_k \mathcal{J}_k = I$ is any cover of the constraint index, then the global feasible set is the intersection of the local ones:*

$$\mathrm{feas}(I) \;=\; \bigcap_k \mathrm{feas}(\mathcal{J}_k).$$

In sheaf language: the functor feas sends colimits (unions) of constraint sets to limits (intersections) of feasible sets, so *local consistency data over any cover glues uniquely to the global feasible set.* On the constraint side of the Galois connection there is no obstruction whatsoever—this is a theorem, not a heuristic. Whatever makes STE hard does not live here.

3 The tractable upper bound: rectangular constraints

Where the hardness *can* live is the variable side. Take $\Xi = \prod_{v \in V} A_v$. Call a constraint *rectangular* (variable-separable, a cylinder in every coordinate) if it has the form $\prod_v P_v$ for some family of per-variable sets $P_v \subseteq A_v$—in Lean, `Set.univ.pi` P. A rectangular constraint inspects each variable independently; it couples nothing.

Theorem 3 (`rectangular_feasibilitySet`)**.** *If every constraint is rectangular, $S_i = \prod_v P_{i,v}$, then the feasible set is itself the rectangle whose v-th side is the pointwise intersection:*

$$\bigcap_i \prod_v P_{i,v} \;=\; \prod_v \Big(\bigcap_i P_{i,v} \Big).$$

This is the representation theorem for the easy case: a fully variable-separable STE problem is represented *exactly* by one subset per variable. The representation lives in $\sum_v |A_v|$ space—linear in the number of variables—never in the $\prod_v |A_v|$ product space. The 2^N wall does not exist for rectangular constraint families.

The cardinality statement is checked as well, in terms of Mathlib's extended natural cardinality encard (which handles infinite sides gracefully):

Theorem 4 (`rectangular_encard`)**.** *For a finite variable set V (`Fintype` V),*

$$\Big| \bigcap_i \textstyle\prod_v P_{i,v} \Big| \;=\; \prod_{v \in V} \Big| \bigcap_i P_{i,v} \Big|,$$

as extended cardinals.

So in the separable case even the *size* of the feasible set is a product of locally-computable sizes: consistency checking, counting, and sampling all decompose per-variable. This is the STE incarnation of the trivial end of CSP tractability: a constraint network with no edges is backtrack-free [6].

4 The obstruction: coupling is not rectangular

The lower bound is a single, minimal counterexample. Let the hypothesis space be two binary variables, $\Xi = (\mathrm{Fin}\, 2 \to \mathrm{Bool})$, and consider the *diagonal* (coupling, equality) constraint

$$\Delta \;=\; \{\, f \mid f(0) = f(1) \,\} \qquad (\texttt{diagonal}).$$

Theorem 5 (`diagonal_not_rectangular`)**.** *There is no family* $t : \mathrm{Fin}\,2 \to \mathcal{P}(\mathrm{Bool})$ *with* $\Delta = \prod_v t_v$.

The proof is the two-line contextuality argument: $(0,0) \in \Delta$ and $(1,1) \in \Delta$, so any rectangle containing Δ has $0 \in t_0$ and $1 \in t_1$, hence contains the mixed point $(0,1)$ —which violates the coupling. The local (per-variable) data of Δ is *full*: projected to either coordinate, everything is possible. The constraint is invisible locally and exists only jointly.

This is precisely the elementary form of the sheaf-theoretic obstruction of Abramsky and Brandenburger [1]: a family of perfectly consistent local sections (here: both values possible at each variable) that admits no factorization through a product—the "global section" data is not determined by, and not recoverable from, the per-variable marginals. In cohomological terms (made precise only in the outlook below), the diagonal carries a nonzero Čech gluing class relative to the cover of V by singletons.

It is also, structurally, the same countermodel this project already uses for a different purpose: `Ste.DynamicFrame.Counterexample` refutes transitivity of possible-coreference (`maySame_not_transitive`) with a three-world model in which pairwise-consistent local identifications fail to assemble into a global one. Pairwise consistency without global consistency is the one phenomenon, seen twice.

Consequences for representation: since rectangles are closed under intersection (`rectangular_feasibilitySet` again), no family of rectangular constraints can ever express Δ. Coupling constraints are therefore *exactly* the constraints that force representations beyond $\sum$-space; the 2^N wall of `card_allExactStates` is built out of them.

5 Block decomposition: $\sum$ across blocks, $\prod$ within

The module `Ste.Decomposition` proves the first positive decomposition result beyond full separability. Suppose the hypothesis space splits into two blocks, $\Xi = X \times Y$, and the constraint family is indexed by a disjoint union $I = \iota \sqcup \kappa$ where ι-constraints depend only on the X coordinate and κ-constraints only on Y (in Lean, `blockFamily` C D takes preimages under the two projections).

Theorem 6 (`feasibilitySet_blockFamily`)**.** *For any block families* $C : \iota \to \mathcal{P}(X)$ *and* $D : \kappa \to \mathcal{P}(Y)$*,*

$$\mathrm{feas}(\mathrm{blockFamily}\ C\ D) \;=\; \mathrm{feas}(C) \times \mathrm{feas}(D).$$

Theorem 7 (`encard_feasibilitySet_blockFamily`, `encard_feasibilitySet_blockFamily_le`)**.** *Consequently the representation cost multiplies,* $|\mathrm{feas}(\mathrm{blockFamily}\ C\ D)| = |\mathrm{feas}(C)| \cdot |\mathrm{feas}(D)|$ *as extended cardinals, and is bounded by* $|X| \cdot |Y|$*.*

Iterated, this is the $\sum$-across-blocks, $\prod$-within-block law: when the constraint hypergraph is disconnected, the exponential lives only inside each connected component. The question becomes: what counts as a block, and what dissolves coupling between blocks? Both answers need a machine-checked notion of *which variables a constraint actually touches.*

6 Support: the scope of a constraint

The module `Ste.Support` defines, for a constraint $T \subseteq \prod_v A_v$ and a set of variables $\sigma \subseteq V$, the predicate `HasSupport` T σ: any two assignments agreeing on σ are both in or both out of T. This is the scope of a constraint in the CSP sense [5], i.e. the hypergraph edge on which the constraint sits. Verified basic theory: supports are monotone (`HasSupport.mono`), every constraint is supported on V (`hasSupport_univ`), support $\emptyset$ holds exactly for the trivial constraints

Ξ and $\emptyset$ (`hasSupport_empty_iff`) — scope-free information is no information — rectangles are supported on their non-trivial coordinates (`hasSupport_pi`), and supports are stable under intersection, so a family with common scope σ has feasible set with scope σ (`HasSupport.inter`, `hasSupport_feasibilitySet`).

The payoff is a sharpening of the coupling obstruction of Section 4 from "not rectangular" to "scope provably $\{0,1\}$":

Theorem 8 (`diagonal_support_full`, `diagonal_hasSupport_pair`)**.** *Every support of the diagonal Δ contains both variables, and $\{0,1\}$ is a support; hence the scope of Δ is exactly $\{0,1\}$. In particular Δ has no support $\emptyset$, $\{0\}$, or $\{1\}$ (`diagonal_not_hasSupport_empty`, `diagonal_not_hasSupport_zero`, `diagonal_not_hasSupport_one`).*

Where `diagonal_not_rectangular` says Δ is not a product of per-variable data, this says more: no proper subset of the variables suffices to decide membership. The coupling core is genuinely two-variable — it is an irreducible edge of the constraint hypergraph.

7 Conditioning: variable elimination, machine-checked

The module `Ste.Conditioning` mechanizes the elimination step of bucket elimination [5]: conditioning a constraint T on fixing one coordinate $v := a$,

$$\mathrm{cond}(T, v, a) \;=\; \{\, f \mid f[v \mapsto a] \in T \,\} \qquad (\texttt{condition}).$$

Machine-checked facts:

Theorem 9 (`condition_feasibilitySet`)**.** *Conditioning commutes with feasibility:* $\mathrm{cond}(\mathrm{feas}(S), v, a) = \mathrm{feas}(i \mapsto \mathrm{cond}(S_i, v, a))$. *Elimination may be performed constraint-by-constraint.*

Theorem 10 (`HasSupport.condition`)**.** *Elimination shrinks scopes: if T has support σ then* $\mathrm{cond}(T, v, a)$ *has support $\sigma \setminus \{v\}$. The eliminated variable leaves the constraint hypergraph.*

Theorem 11 (`condition_pi`; `condition_condition_self`, `condition_condition_comm`)**.** *The rectangular class is closed under conditioning on a feasible value (the v-th side becomes trivial), and elimination is idempotent per variable and commutes across distinct variables — the elimination order is immaterial.*

Theorem 12 (`condition_diagonal_rectangular`, `condition_diagonal_hasSupport_one`)**.** *Conditioning the diagonal on either value of variable* 0 *yields a* rectangular *constraint of scope* $\{1\}$ *— although the diagonal itself is provably not rectangular and provably has no support* $\{1\}$*. Fixing one variable of the coupling core moves it from the intractable class to the tractable one.*

Finally, the cut-variable theorem connects elimination to the block decomposition of Section 5. Say the family S is a *block family* for the partition $\{\sigma, \sigma^{\mathsf{c}}\}$ if every constraint has support inside σ or inside σ^{c}; and say v is a *cut variable* if every constraint has support inside $\sigma \cup \{v\}$ or $\sigma^{\mathsf{c}} \cup \{v\}$ — the two sides interact only through v.

Theorem 13 (`piecewise_mem_feasibilitySet`)**.** *The feasible set of a block family is closed under block-wise recombination: if $f, g \in \mathrm{feas}(S)$ then the splice σ.*piecewise *$f\ g$ (equal to f on σ, to g off σ) is again in* $\mathrm{feas}(S)$*. This is the pi-space form of "*$\mathrm{feas}(S)$ *is a product of two independent subproblems" (cf. `feasibilitySet_blockFamily`).*

Theorem 14 (`condition_cut_blocks`, `piecewise_mem_condition_feasibilitySet`)**.** *Conditioning on a cut variable disconnects the problem: after fixing $v := a$, every conditioned constraint has support inside a single block, and the conditioned feasible set* $\mathrm{cond}(\mathrm{feas}(S), v, a)$ *is closed under block-wise recombination.*

This is variable elimination confining the exponential: eliminating a cut vertex of the constraint hypergraph splits the remaining problem into independent subproblems, each paying only its own product cost. The next section makes the cut theorem general and the cost per bag quantitative.

8 Cut elimination and treewidth: the bag invariant

The module `Ste.Treewidth` proves, over an abstract variable type V with alphabets A_v, the two accounting facts that underlie treewidth bounds [5], then the quantitative per-bag cost.

Scope accounting

Theorem 15 (`HasSupport.inter_union`; `HasSupport.iInter`)**.** *Joins union scopes: if T has support σ and U has support τ, then $T \cap U$ has support $\sigma \cup \tau$; for a family, $\bigcap_i T_i$ has support $\bigcup_i \sigma_i$.*

Theorem 16 (`condition_inter`)**.** *Conditioning distributes over intersection:* $\mathrm{cond}(T \cap U, v, a) = \mathrm{cond}(T, v, a) \cap \mathrm{cond}(U, v, a)$*. Elimination commutes with joining.*

With `HasSupport.condition` (elimination removes v from every scope), the scope of an intermediate table of bucket elimination is $(\bigcup_i \sigma_i) \setminus \{v\}$ — the *bag* of the step.

Cut elimination, general form

Theorem 17 (`cut_elimination`)**.** *If T, U have supports σ, τ with $\sigma \cap \tau \subseteq \{v\}$ (the constraints share only the cut variable v), then for any value a the conditioned constraints* $\mathrm{cond}(T, v, a)$, $\mathrm{cond}(U, v, a)$ *have* disjoint *supports $\sigma \setminus \{v\}$ and $\tau \setminus \{v\}$.*

Theorem 18 (`inter_nonempty_iff_of_disjoint_support`; `condition_inter_nonempty_iff`)**.** *Disjoint scopes make joint feasibility independent: for supports $\sigma \perp \tau$, $T \cap U \neq \emptyset$ iff $T \neq \emptyset$ and $U \neq \emptyset$ (splice the witnesses along σ). Consequently, if T, U share only the cut variable v, then after conditioning on v the joint problem is satisfiable iff each side is satisfiable separately. This is the pi-space form of the block product `feasibilitySet_blockFamily`: fixing a cut variable dissolves the join into independent checks.*

The bag bound

For a scope $\sigma \subseteq V$, the *table* of a constraint T is its restriction image $\mathrm{table}(\sigma, T) = \{\, f|_\sigma \mid f \in T \,\}$ (`table`) — the relation bucket elimination materializes.

Theorem 19 (`HasSupport.eq_preimage_table`)**.** *A constraint with support σ is the preimage of its σ-table: the table is a faithful representation, losing no information.*

Theorem 20 (`table_encard_le`; `table_encard_le_pow`)**.** *For finite σ and finite alphabets, the table of* any *constraint satisfies* $|\mathrm{table}(\sigma, T)| \leq \prod_{v \in \sigma} |A_v|$*; if moreover $|A_v| \leq k$ for all v and $|\sigma| \leq w+1$, then* $|\mathrm{table}(\sigma, T)| \leq k^{w+1}$.

Theorem 21 (`elimination_step`)**.** *The elimination step of bucket elimination—join constraints with scopes σ_i, then eliminate v—produces a constraint that (i) has support the bag $(\bigcup_i \sigma_i) \setminus \{v\}$, (ii) equals the preimage of its bag table, and (iii) has bag table of at most k^{w+1} rows whenever the bag has at most $w+1$ variables over alphabets of size at most k.*

This is the per-step space invariant of variable elimination: an elimination whose every bag stays within $w+1$ variables never materializes more than k^{w+1} rows at a step — the induced-width cost a^{w^*+1} [5].

The elimination-order total-space bound

The global statement of bucket elimination sums the bag bound along a whole elimination *order*. Presenting the order by the list of its bags:

Theorem 22 (`elimination_order_total_bound`)**.** *Let* bags *be a list of finite bags, each of at most $w+1$ variables, over alphabets of size at most $k \geq 1$. Then the total capacity of all bag tables satisfies*

$$\sum_{\sigma \in \text{bags}} \prod_{v \in \sigma} |A_v| \;\leq\; |\text{bags}| \cdot k^{w+1}.$$

Theorem 23 (`elimination_order_table_total_bound`; `table_encard_le_pow_finset`)**.** *For any list of realized steps (σ_i, T_i) with $|\sigma_i| \leq w+1$, the* actual *materialized sizes satisfy $\sum_i |\text{table}(\sigma_i, T_i)| \leq n \cdot k^{w+1}$ where n is the number of steps.*

This is the honest "bucket elimination runs in $n \cdot a^{w+1}$ total space" theorem, taking the order and its width as input.

9 Running the order: the elimination fold, machine-checked

The module `Ste.Elimination` mechanizes *running* a whole elimination order. An order is a list of (v, a_v) steps; `eliminate` folds conditioning over it, and `condition_eq_self` shows conditioning outside the scope is trivial (so a step only touches the bucket of its variable).

Theorem 24 (`HasSupport.eliminate`; `mem_eliminate_iff`; `eliminate_eq_univ_or_empty`)**.** *Running an order removes every eliminated variable from the support: the residue of a σ-supported constraint is supported on $\sigma \setminus$* eliminated*. Elimination is substitution: f survives the fold iff f overwritten with the eliminated values lies in the original constraint. Hence a* complete *order (one covering σ) leaves a decision: the residue is Ξ or $\emptyset$.*

The full data-structure fold operates on a list of constraints with finite scopes. One `bucketStep` at (v, a) collects the bucket of constraints whose scope contains v, joins it — the join is supported on the union of the scopes (`hasSupport_joinConstraint`) — conditions on v, and keeps the rest untouched.

Theorem 25 (`bucketEliminate_support`; `bucketEliminate_scope_subset`; `joinConstraint_bucketEliminate`
Along the whole run: every live constraint stays supported on its recorded scope; every live scope avoids all eliminated variables; and the joint constraint of the state is exactly the elimination fold of the original joint constraint — the data structure is a faithful implementation of variable elimination.

Theorem 26 (`bucketEliminate_decides`)**.** *Run a complete order — one whose variables cover every initial scope. Then every surviving constraint has empty scope, hence is* Ξ *or* $\emptyset$*, and the joint residue — the full elimination of the original joint constraint — is itself* Ξ *or* $\emptyset$*: bucket elimination terminates in a decision.*

Theorem 27 (`bucketEliminate_total_space`)**.** *Let* bucketBags *trace the (bag, constraint) pair each step materializes (n steps materialize n tables,* ***`length_bucketBags`****, each faithfully tabled on its bag,* ***`bucketBags_support`*** *with* ***`HasSupport.eq_preimage_table`****). If every materialized bag has at most* $w+1$ *variables — the order has width* w *on this instance — then over alphabets of size at most* k *the total size of all materialized tables is at most* $n \cdot k^{w+1}$*.*

Together: a width-w elimination order decides feasibility of the substituted problem in $n \cdot a^{w+1}$ total table space. One honest caveat, stated plainly: `condition` *substitutes* a chosen value, so the fold decides the problem *conditioned on the chosen values* (a_v) (`mem_eliminate_iff` makes this exact). The *projective* step of bucket elimination — existentially quantifying v rather than substituting it, so that a single run decides unconditional feasibility — is *not yet proven* (outlook), as are: a tree-decomposition datatype with its junction-tree representation theorem, and the `encard` factorization of the conditioned feasible set through restriction maps (the quantitative analogue of `encard_feasibilitySet_blockFamily`).

The width hypothesis discharged: induced treewidth

The width w above is a hypothesis on the *given* order. `Ste.Treedecomp` removes it. Say the instance B *achieves width* w (`AchievesWidth`) if some complete elimination order — one whose eliminated variables cover every scope — materializes only bags of at most $w+1$ variables on B. Over a finite variable set with inhabited alphabets the predicate is satisfiable: the trivial order eliminating every variable once (`elimAll`) is complete (`eliminated_elimAll`), and every bag it materializes is a finite set of variables, so every instance achieves width $|V|$ (`achievesWidth_card`).

Theorem 28 (`inducedTreewidth`; `achievesWidth_inducedTreewidth`; `inducedTreewidth_le`)**.** *The* induced treewidth $\mathrm{tw}(B)$ *of an instance* B *is the least achievable width. It is well-defined (the minimum of a nonempty set of naturals,* $\mathrm{tw}(B) \le |V|$ *by* ***`inducedTreewidth_le_card`****) and* attained*: a concrete optimal elimination order realizes it.*

Theorem 29 (`bucketEliminate_treewidth_bound`)**.** *For every instance* B *of scoped constraints over alphabets of size at most* k*, there* exists *a complete elimination order that decides the substituted joint constraint — the elimination residue is* Ξ *or* $\emptyset$ *— while materializing at most* $n \cdot k^{\mathrm{tw}(B)+1}$ *total table rows,* n *the length of the order. The width hypothesis of* ***`bucketEliminate_total_space`*** *is discharged: the bound holds at the instance's own attained induced treewidth, with no free width parameter.*

Two former limits of this discharge, both now resolved. First, the substitution caveat — that the run decides feasibility only *conditioned on the chosen values* — is removed by projective elimination (§11.1): eliminating by projection instead of substitution decides *unconditional* feasibility. Second, $\mathrm{tw}(B)$ is elimination width (Dechter's induced width over all complete orders); its *equivalence* with the tree-decomposition treewidth of the primal graph is now *fully* mechanized in both directions (next section, `treewidth_primalGraph_eq`). What remains genuinely out of reach is only *efficient* computation of $\mathrm{tw}(B)$: the optimal order is obtained by minimization over a nonempty set, and computing treewidth is NP-hard [2] — a settled hardness result, not a gap in this development.

10 The graph-treewidth bridge, machine-checked

The induced treewidth $\mathrm{tw}(B)$ of the previous section is defined operationally, through elimination orders. Its classical counterpart is the Robertson–Seymour treewidth of the *primal* (co-occurrence) graph G_B — vertices are variables, and $u \sim v$ iff some scope of B contains both. The file `Ste.GraphTreewidth` builds the graph side from scratch (Mathlib has `SimpleGraph` but no tree decompositions) and proves one direction of the equivalence.

A `TreeDecomposition` of a graph G is a list of bags with a strictly increasing parent function on positions, satisfying the three Robertson–Seymour axioms: every vertex lies in a bag (`vertexCover`), every edge lies in a common bag (`edgeCover`), and running intersection (`runningIntersection`), stated in parent-closure form — a vertex in bag i and in some later bag lies in bag $\mathrm{parent}(i)$. For an increasing parent this is equivalent to the usual "bags containing a vertex form a connected subtree": the parent-walk from any occurrence strictly climbs, stays among occurrences, and terminates at the topmost one, which is their common ancestor. The width is the largest bag size minus one, and

$$\mathrm{treewidth}(G) \;:=\; \inf\{\, w \mid \exists\, \text{tree decomposition of width} \le w \,\},$$

well-defined and attained because the single-bag decomposition $[\,V\,]$ always exists over a finite vertex set (`TreeDecomposition.single`, `treewidth_le_card`).

Theorem 30 (`primalGraph_elimination_cover`)**.** *For every instance B over a finite variable set with inhabited alphabets there exists a list of bags that covers every variable and every primal edge of B, each of size at most $\mathrm{tw}(B)+2$.*

The bags are the *elimination bags*: at each step of the optimal order, the recorded message scope $\beta(v)$ together with the eliminated variable v itself. This is exactly the elimination clique $\{v\} \cup \mathrm{nbr}(v)$ of the induced graph.

Theorem 31 (`treewidth_primalGraph_le`)**.** *For every instance B over a finite variable set with inhabited alphabets,*

$$\mathrm{treewidth}(G_B) \;\le\; \mathrm{tw}(B)+1.$$

The proof exhibits a genuine tree decomposition of G_B — all three axioms, running intersection included, discharged sorry-free. The construction dedupes the optimal order to a repetition-free one with a sublist of bags (`exists_nodup_order`, using that empty-scope constraints are `InertLe` for elimination), pads it to eliminate every variable (the padding bags are empty, since the state is already decided), and builds the decomposition recursively (`exists_elim_treeDecomposition`): the bag of each step is $\{v\} \cup \beta(v)$, and its parent is a later bag containing its message scope. Running intersection closes because a deduped order never re-eliminates a variable, so each eliminated variable appears in no bag above its own.

The offset is exactly one, and it is not an artifact

$\mathrm{treewidth}(G_B) \le \mathrm{tw}(B)+1$, not equality of the two symbols, because they count *different* things by exactly one variable. Our `inducedTreewidth` records the *message scope* $\beta(v)$, which excludes the eliminated variable v; a Robertson–Seymour bag must include v (otherwise the edges incident to v are uncovered). So the elimination bag is one larger than the message scope, and:

$$\max_v |\beta(v)| \;=\; \mathrm{tw}(G_B) \quad \text{(Dechter's induced width)}, \qquad \mathrm{tw}(B) \;=\; \max_v |\beta(v)| - 1.$$

Read symbolically: the quantity equal to graph treewidth is $\max_v |\beta(v)|$ (the largest message scope), while our slack parameter `inducedTreewidth` equals $\mathrm{tw}(G_B) - 1$ for any coupled instance. For a *fixed* order the induced width is $\geq \mathrm{tw}(G_B)$ (GTE); minimized over orders they are equal — the classical Bodlaender/Dechter theorem [3, 5].

Both halves are now machine-checked. The forward (tractable) half, `treewidth_primalGraph_le` above, turns an elimination order of width w into a tree decomposition of width $\leq w+1$. The converse — the deep half, that no tree decomposition beats elimination — is `Ste.TreewidthConverse`: from any tree decomposition of width w we *extract* a complete elimination order of message-scope $\leq w$ by eliminating variables in ascending order of their topmost bag position. The combinatorial heart is the chain lemma `TreeDecomposition.mem_top_of_shared_bag` (if u and x share a bag and $\mathrm{top}\, u \leq \mathrm{top}\, x$ then x lies in u's topmost bag — the rooted-tree avatar of running intersection), which drives `bucketBags_card_le_of_pairwise_top`: eliminating in ascending-top order keeps every message scope inside $\mathrm{bags}[\mathrm{top}\, u] \setminus \{u\}$, of size $\leq w$. Hence `inducedTreewidth_le_treewidth_sub_one` ($\mathrm{tw}(B) \leq \mathrm{tw}(G_B) - 1$, unconditional) and, whenever the primal graph has an edge,

Theorem 32 (`treewidth_primalGraph_eq`)**.** $\mathrm{tw}(G_B) = \mathrm{tw}(B) + 1$.

The edge hypothesis is sharp: an edgeless instance has $\mathrm{tw}(G_B) = \mathrm{tw}(B) = 0$, so the unconditional statement is `max_treewidth_primalGraph_one_eq`: $\max(\mathrm{tw}(G_B), 1) = \mathrm{tw}(B) + 1$. The equivalence of Robertson–Seymour treewidth and bucket-elimination induced width is thus fully mechanized, no longer outlook.

11 Four more leads, run down

The breadth-first sweep of the remaining "outlook" claims closed four further leads, each machine-checked sorry-free.

11.1 Projective elimination: unconditional feasibility

The conditioning fold decides feasibility only relative to the substituted values. Projective elimination (`Ste.Projection`) removes that caveat. Define $\mathrm{project}(T, v) = \{f \mid \exists a,\ f[v \mapsto a] \in T\}$: eliminate v by projecting it out, not by fixing a value.

Theorem 33 (`project_nonempty_iff`)**.** $\mathrm{project}(T, v) \neq \emptyset \iff T \neq \emptyset$.

The exists-step preserves nonemptiness *exactly* (given $g \in T$, take the value $g(v)$; the reverse is immediate). Folding over a complete elimination list therefore decides unconditional feasibility (`projectEliminate_decides_feasibility`, `projectEliminate_joinConstraint_decides`): the residue is Ξ iff the instance is feasible and $\emptyset$ iff it is not, with no chosen values. The exists-step is the union of all conditionings (`project_eq_iUnion_condition`), tying it to the existing machinery.

11.2 Quantitative factorization over a partition

The binary block law generalizes to arbitrarily many independent blocks (`Ste.Factorization`). For a finite index ι and per-block constraints C_i, the feasible set is $\prod_i C_i$ (as `Set.pi`) and

Theorem 34 (`encard_feasibilitySet_blocks`)**.** $\left|\mathrm{feas}\right| = \prod_i |C_i|$

so the exponential representation cost is confined to each coupled block — an estimator pays $\sum_i \mathrm{cost}(C_i)$ across blocks, not $\prod_i$ for the joint set. This is the $\sum$-across, $\prod$-within law made quantitative at the cardinality level.

11.3 The variable-side presheaf and its gluing dichotomy

An earlier section proved gluing on the *constraint* poset; the open direction was the *variable* side. `Ste.VariablePresheaf` assigns to each variable set W the local sections localSections(T, W) with functorial restriction (`restrict_restrict`), a genuine presheaf. Its sheaf/gluing condition over the singleton cover of V *succeeds* for rectangular constraints — `rectangular_glue` gives a *unique* global glue — and provably *fails* for the coupling: `diagonal_gluing_fails` exhibits local sections (false at x, true at y), each individually extendable to the diagonal, that no global diagonal section restricts to. The glue of the diagonal's singletons has four points to the diagonal's two (`singletonGlue_diagonal_encard`). For the singleton cover the characterization is in fact an *iff* — gluing succeeds precisely for rectangles (`cechVanishes_iff_rectangular`, next-but-one section); the general-cover version (gluing precisely on the tractable fragments) stays outlook.

11.4 The concrete Čech obstruction

Following Abramsky and Brandenburger [1], the obstruction to gluing compatible local sections into a global one is a Čech $\check{H}^1$ class. `Ste.CechObstruction` computes it concretely for the singleton cover: with compatible(T) the rectangle of T's projections and glued$(T) = T$, the obstruction is cechObstruction$(T) = |\,\text{compatible}(T)| - |\,\text{glued}(T)|$.

Theorem 35 (`rectangular_cechObstruction`; `diagonal_cechObstruction`)**.** *The obstruction vanishes for every rectangular constraint ($\check{H}^1 = 0$) and equals $2 \neq 0$ for the diagonal.*

Vanishing forces rectangularity (`rectangular_of_cechVanishes`), bridging the cohomological obstruction to the representation obstruction of the coupling. The general theory (arbitrary covers, the full $\check{H}^1$ functor, the quantitative "obstruction size = representation blow-up" claim) remains cited framework and outlook.

11.5 The junction-tree representation

`Ste.JunctionTree` packages the bucket-elimination trace as one projected bag table per eliminated bucket. The representation is *faithful* — each materialized constraint is the preimage of its bag table (`junctionTables_faithful`) — and

Theorem 36 (`junctionTree_size_le`; `junctionTree_size_linear`)**.** *Some complete order decides feasibility with a faithful bag-table representation of total size $\leq n \cdot k^{\mathrm{tw}(B)+1}$; at fixed width w and alphabet bound k this is $n \cdot C$ with $C = k^{w+1}$ a constant — linear in the number of variables.*

This is the tractable direction of the CSP dichotomy [5, 6, 8]: bounded treewidth yields a polynomial-size exact representation of a feasible set that may itself be exponential. The matching lower bound is the next section.

12 The dichotomy's hard side: unbounded coupling is exponential

The tractable direction above is only half a dichotomy without a matching lower bound. `Ste.CouplingLowerBound` supplies the concrete hard side. First it upgrades the presheaf gluing dichotomy of the variable-side presheaf (above) to a genuine *iff* for the singleton cover:

Theorem 37 (`cechVanishes_iff_rectangular`)**.** *The Čech obstruction of the singleton cover vanishes — equivalently, every compatible family of local sections glues — iff the constraint is a rectangle* univ.pi P*.*

So coupling is *exactly* the obstruction to singleton-cover gluing. Then it scales the two-bit diagonal to the *n-fold all-equal coupling* $\Delta_n = \{f : \mathrm{Fin}\, n \to \alpha \mid \forall i\, j,\ f_i = f_j\}$ — the maximal coupling, whose primal graph is complete (induced width $n-1$). Its feasible set is tiny (`allEqual_encard`: $|\Delta_n| = |\alpha|$, one constant vector per value), yet every margin is full, so:

Theorem 38 (`cechObstruction_allEqual`; `cechObstruction_allEqual_ge`)**.** cechObstruction$(\Delta_n) = |\alpha|^n - |\alpha|$*, and for* $|\alpha| \geq 2$ *this is* $\geq 2^n - 2$*.*

The gap between the $|\alpha|$ genuine solutions and the $|\alpha|^n$ margin-compatible families grows exponentially in the number of variables (`allEqual_not_rectangular`, `allEqual_not_cechVanishes` record non-rectangularity at every arity). Set against `junctionTree_size_linear`'s $n \cdot k^{w+1}$ at bounded width, this is the STE form of the bounded-width tractability dichotomy [5], now machine-checked on both sides: bounded coupling is linear, unbounded coupling is exponential.

13 Local consistency: the width-1 tractable case

The constructive counterpart of bounded width is that a locally consistent, acyclic instance is *solvable* — Freuder's backtrack-free theorem [6]. `Ste.Consistency` mechanizes the chain (width-1) case. For a path of variables $\mathrm{Fin}(n+1)$ with domains D_i and binary constraints R_i between consecutive variables, *arc consistency* asks that every domain value have a forward and a backward support.

Theorem 39 (`arcConsistent_chain_solvable`; `arcConsistent_backtrackFree`)**.** *A nonempty arc-consistent chain has a global solution* f *with* $f_i \in D_i$ *and* $R_i(f_i, f_{i+1})$ *for every edge; moreover* every *value of* every *domain lies on such a solution — extended rightward by forward supports and leftward by backward supports, with no choice ever retracted.*

The solution set is the 'feasibilitySet' of the chain constraints (`chainSet_eq_feasibilitySet`), and arc consistency is the statement that the two-variable edge sections cover the margins — the concrete, width-1 instance of "k-consistency as higher gluing." The general (forest, and higher-width k-consistency) cases remain outlook.

14 Outlook: the cohomological program (not proven)

Everything in this section is conjecture or programme. None of it is machine-checked, and none of it should be cited as a result of this project.

Conjecture (outlook) (Feasibility as a sheaf on a site over variables)**.** *Section 2 proved gluing over the* constraint *poset. The substantive open direction is the* variable *side: assign to each subset* $W \subseteq V$ *of variables the set of partial assignments on* W *consistent with all constraints whose support lies in* W*. We conjecture this is a presheaf on a Grothendieck site over* $\mathcal{P}(V)$ *whose sheaf condition, relative to a cover of* V *by constraint supports, holds precisely for the tractable fragments; the diagonal example shows the presheaf is not a sheaf for the singleton cover in general.*

Conjecture (outlook) (k-consistency is higher gluing)**.** *Local (*k*-)consistency in the CSP sense [5, 6]—every consistent assignment on* $k-1$ *variables extends to any* k*-th—should be exactly the statement that sections glue along the cover of* V *by* k*-element subsets. Freuder's theorem (strong* k*-consistency plus width* $< k$ *implies backtrack-free search) would then become: on covers refined enough for the treewidth, the presheaf* is *a sheaf, and a global section is constructed greedily.*

Conjecture (outlook) ($\check{H}^1$ measures intractability)**.** *Following Abramsky and Brandenburger [1], the obstruction to a family of compatible local sections extending to a global one should be captured by a Čech cohomology class in $\check{H}^1$ of the support cover with coefficients in the (relative) presheaf of partial solutions; the class vanishes iff gluing succeeds. The conjectured quantitative version: the difficulty of an STE instance—the size blow-up of its smallest exact representation—is controlled by the size of this obstruction, with rectangular families the $\check{H}^1 = 0$ case of Section 3 and the diagonal of Section 4 the minimal nonvanishing class.*

Conjecture (outlook) (Tractability dichotomy)**.** *Bounded treewidth of the constraint (co)occurrence structure—or, conjecturally equivalent in this setting, bounded obstruction in the sense above—yields a polynomial-size glued representation (junction-tree style: rectangles over bags, glued along separators), and unbounded coupling reproduces the 2^N wall. This would be the STE form of the classical CSP dichotomy between bounded-width tractability and general intractability [5].*

What stands *proven* against this programme: the fully separable case is exactly linear (`rectangular_feasibili` `rectangular_encard`); one coupling suffices to leave it, and its scope is provably irreducible (`diagonal_not_rectangular`, `diagonal_support_full`); disconnected blocks multiply (`feasibilitySet_blockFa` `encard_feasibilitySet_blockFamily`); eliminating a variable removes it from every scope (`HasSupport.conditio` eliminating a *cut* variable disconnects the residual problem into independent blocks with disjoint scopes (`piecewise_mem_condition_feasibilitySet`, `cut_elimination`) whose joint feasibility factors into independent checks (`condition_inter_nonempty_iff`); every single elimination step is confined to its bag with table cost at most $\prod_{v\in\mathrm{bag}} |A_v| \le k^{w+1}$ (`elimination_step`, `table_encard_le`); a whole width-w order of n steps costs at most $n \cdot k^{w+1}$ total table space (`elimination_order_total_bound`, `bucketEliminate_total_space`); and the bucket-elimination fold itself is verified correct and terminating in a decision for the substituted problem (`joinConstraint_bucketElim` `bucketEliminate_decides`)—the width-1 case of the conjectured dichotomy, plus its space accounting, plus a verified solver skeleton; and the width hypothesis is discharged: the induced treewidth of an instance is well-defined and attained by a concrete optimal order (`achievesWidth_inducedTreewidth`), yielding the unconditional $n{\cdot}k^{\mathrm{tw}(B)+1}$ decision-and-space bound (`bucketEliminate_treewidth_bound`). The constraint-side gluing (`feasibilitySet_eq_iInter_cover`) guarantees the difficulty is genuinely about variables, not about how information is indexed. Open in this direction: projective ($\exists$-)elimination, the quantitative `encard` factorization of the conditioned feasible set over a partition, the equivalence of elimination width with tree-decomposition treewidth, junction trees, efficient (or any) computation of $\mathrm{tw}(B)$, and the full treewidth dichotomy.

References

[1] Samson Abramsky and Adam Brandenburger. The sheaf-theoretic structure of non-locality and contextuality. *New Journal of Physics*, 13:113036, 2011. doi: 10.1088/1367-2630/13/11/113036. arXiv:1102.0264.

[2] Stefan Arnborg, Derek G. Corneil, and Andrzej Proskurowski. Complexity of finding embeddings in a k-tree. *SIAM Journal on Algebraic and Discrete Methods*, 8(2):277–284, 1987. doi: 10.1137/0608024.

[3] Hans L. Bodlaender. A partial k-arboretum of graphs with bounded treewidth. *Theoretical Computer Science*, 209(1–2):1–45, 1998. doi: 10.1016/S0304-3975(97)00228-4.

[4] Patrick L. Combettes. The foundations of set theoretic estimation. *Proceedings of the IEEE*, 81 (2):182–208, 1993. doi: 10.1109/5.214546. Retrieved from the author's page, `https://pcombet.math.ncsu.edu/proc.pdf`.

[5] Rina Dechter. *Constraint Processing*. Morgan Kaufmann, San Francisco, 2003.

[6] Eugene C. Freuder. A sufficient condition for backtrack-free search. *Journal of the ACM*, 29 (1):24–32, 1982. doi: 10.1145/322290.322292.

[7] Bernhard Ganter and Rudolf Wille. *Formal Concept Analysis: Mathematical Foundations*. Springer-Verlag, Berlin, Heidelberg, 1999. Translated by Cornelia Franzke.

[8] Martin Grohe. The complexity of homomorphism and constraint satisfaction problems seen from the other side. *Journal of the ACM*, 54(1):1–24, 2007. doi: 10.1145/1206035.1206036.